MigrRDMA: Enabling RDMA Live Migration in the Software

Xiaoyu Li[♣] Weizhe Zhang[♣] Fengyuan Ren[♥★]

[♣]Pengcheng Laboratory, Shenzhen, China
[♥]Tsinghua University, Beijing, China

***Abstract*—Live migration is critical to ensure services are not interrupted during host maintenance in data centers. On the other hand, RDMA has been widely adopted in data centers, and has attracted both academia and industry for years. However, live migration of RDMA is not supported in today's data centers. Although modifying RDMA NICs (RNICs) to be aware of live migration has been proposed for years, it relies on extra hardware support. This paper proposes `MigrRDMA`, a software-based RDMA live migration system. `MigrRDMA` provides a software indirection layer to achieve transparent switching to new RDMA communications. Unlike previous RDMA virtualization that provides sharing and isolation, `MigrRDMA`'s indirection layer focuses on keeping the RDMA states on the migration source and destination identical from the perspective of applications. We implemented the `MigrRDMA` prototype over Mellanox RNICs. Our evaluation shows that `MigrRDMA` adds little downtime when migrating a container with live RDMA connections running at line rate. Besides, the `MigrRDMA` virtualization layer only adds 2% ∼ 9% extra overhead in the data path. When migrating Hadoop tasks, `MigrRDMA` only incurs an extra 3-second job completion time.**

***Index Terms*—Data Center, RDMA, Live Migration, Virtualization.**

I. Introduction

LIVE migration enables server upgrades, server maintenance, and load balancing without interrupting the running services. It has been widely adopted for virtual machines (VMs) [1]–[4], containers [5] and bare-metal instances [6], [7]. Meanwhile, modern data centers have been widely deploying Remote Direct Memory Access (RDMA) to accelerate distributed applications, such as machine learning [8], [9], distributed storage [10], [11], databases [12], [13], etc. As RDMA is running in the clouds, RDMA live migration is now one of the critical technologies in modern data centers.

However, today's data centers do not support live migration of RDMA. The main reason is that the majority of RDMA states are maintained by RNICs due to the hardware offloading nature. These states are not available externally, thus unable to be migrated to another server's RNIC. Existing works [14]–[16] modify the RNICs to enable RDMA live migration. They rely on extra hardware supports. This paper aims to provide a software-based RDMA live migration solution that can be readily deployed over commodity RNICs.

Nevertheless, realizing software-based RDMA live migration is not straightforward due to the following challenges. First, setting up RDMA communication is slow [17]. To reduce the blackout time, it is necessary to adopt communication pre-setup during memory pre-copy. Pre-copy is a live migration phase when the migrated service is partially restored on the migration destination and is still running on the migration source. Nevertheless, for ease of restoring the memory pages, live migration tools may map the application's memory temporarily at a location other than the application's original address. As the memory structure during pre-copy is different, it is difficult to restore the registered memory regions during pre-copy. Second, during communication, applications need to use the states managed by RNICs to identify different RDMA resources and to verify memory access authorization. The workflow bypasses the kernel. Thus, it is challenging to hide the differences between the old and new RDMA communications with a software-based solution. Third, RNICs still continue processing the work requests during live migration. The software is unable to get the states of inflight work requests from RNICs. It is hard to keep the execution of the work requests consistent using an interposing layer.

In this paper, we provide `MigrRDMA`, a software-based live migration system for RDMA-based applications running on commodity RNICs. `MigrRDMA` introduces a software indirection layer in the RDMA driver, and mainly provides three features to support RDMA live migration. First, `MigrRDMA` explores two different methods to restore the RDMA-related memory structures. One is to directly map the RDMA-related memory structures to the original addresses, and use the original addresses to register the MRs, which requires modification to the existing memory restoration workflows. The other is to use the temporary virtual addresses to restore the MRs during pre-copy. When applications post WRs, `MigrRDMA` translates the original addresses to the ones used for MR registration. Second, `MigrRDMA` provides efficient virtualization of the RDMA communications' states. The RDMA driver manages the translation table of virtual to physical values, and shares the table with the RDMA library. The RDMA library does the translation inside the userspace. For translation efficiency, `MigrRDMA` assigns the virtual access keys one by one, and uses an array to maintain the translation table. As such, the RDMA library can translate the states with low overheads. Third, `MigrRDMA` proposes wait-before-stop to handle inflight work request consistency. `MigrRDMA` suspends the RDMA communication, and waits for the completion of inflight work

The manuscript was received on 28 Nov. 2025, revised on 16 Apr. 2026, accepted on 12 May 2026 with the approval by IEEE/ACM TRANSACTIONS ON NETWORKING Editor M. Tornatore, and published on 18 May 2026. This work was supported in part by the Major Key Project of Pengcheng Laboratory (No. PCL2025A07), the National Key Research and Development Program of China (No. 2022YFB2901404), and the National Natural Science Foundation of China (NSFC) under Grant No. 62132007 and No. 62221003. Part of this work has been published in SIGCOMM 2025 [DOI: 10.1145/3718958.3750487].

★ Fengyuan Ren is the corresponding author.

requests. Although the communication is suspended, applications can still execute computation tasks. After restoration, the indirection layer acts as a virtual RNIC to let applications process all inflight CQ entries.

We implement `MigrRDMA` virtualization features in the Mellanox OFED driver and library. We also implement a CRIU [18] plugin and extend runc [19] to integrate our solution with the existing container live migration workflow. Evaluation results demonstrate that `MigrRDMA` adds little downtime when migrating a container with RDMA connections running at line rate, while achieving low virtualization overheads, only 2% $\sim$ 9% extra overheads in the data path. `MigrRDMA` only incurs an extra 3 seconds in the job completion time when migrating a Hadoop task. Our prototype is available at https://github.com/hittingnote/migrrdma.

In conclusion, our main contributions of this paper include:

- Proposing the first software-based system to support RDMA live migration in data centers, which provides three main ideas to support RDMA live migration purely in the software, namely, communication pre-setup, efficient virtualization of RDMA communication states, and wait-before-stop.
- Implementing a prototype of `MigrRDMA`, which integrates well with the existing container live migration system.
- Conducting evaluation on `MigrRDMA`, and the results show the great benefit from RDMA pre-setup, correctness and negligible overheads of wait-before-stop, and low overheads introduced in the data path.

The rest of the paper is organized as follows: §II introduces the background on live migration, and states our motivation and challenges of realizing software-based RDMA live migration. §III proposes the design of `MigrRDMA`. §IV implements the prototype of `MigrRDMA`, which supports RDMA live migration for containers. §V evaluates `MigrRDMA` in migration blackout time, the overhead of wait-before-stop, and the performance implications `MigrRDMA` incurs. §VI introduces related work. §VII summarizes this paper.

Ethic: This work does not raise any ethical issues.

II. Background and Motivation

In this section, we provide a brief background on the live migration of virtual machines, containers, and bare-metal instances. Then, we state our motivation for proposing software-based RDMA live migration.

A. Live Migration

Live migration is the procedure of moving running services from one physical server to another with minimal disruption. It is one of the key technologies behind the compute engine products of modern data centers [1] to achieve server upgrades, data center maintenance, and load balancing during the service runtime. Live migration has been widely adopted for VMs [1]–[4]. There are also trends of enabling live migration of containers [5], [20] and even bare-metal instances [6], [7]. For example, Google [5] has enabled container live migration in data centers.

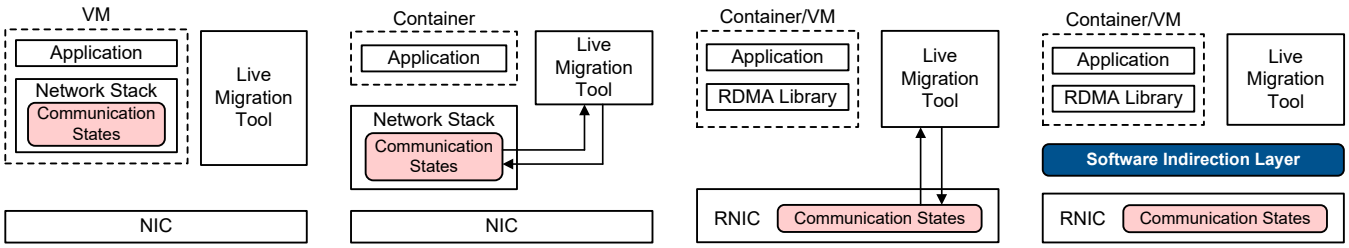
To keep transparent to the running services, live migration tools need to migrate and restore the internal states of the services. These states mainly include memory, storage, and networking. The memory/storage states contain two parts, i.e., virtual address tables and the contents of all pages. For VMs and containers, live migration tools map the same virtual addresses to new physical addresses, and write exactly the same contents to all the regions. For bare-metal instances, the guest OSs need to run lightweight hypervisors to introduce lightweight virtualization [21], [22] or even on-demand virtualization [6], [7]. The rest of the migration workflow is similar to that of VMs. The states of the memory/storage are typically very large. Therefore, directly adopting a stop-and-copy approach (stop the running services, copy the states, and restore the services) will incur too much blackout time. There are two approaches to reduce the blackout time, namely, pre-copy and post-copy.

For pre-copy [2], live migration tools copy the states of memory/storage to the migration destination and start restoring the services, while the services are still running on the migration source. The pre-copy is done iteratively. The first iteration copies all the memory/storage, and each subsequent iteration only copies the differences from the previous one. Upon each iteration, live migration tools on the migration destination merge the new differences with the previously restored states. The pre-copy terminates when the size of the modified pages is below a threshold, or when the bandwidth required to transfer the modified pages is so great that the running services may experience performance declines due to the network contention [2]. Then, live migration tools migrate and restore the rest pages during stop-and-copy.

For post-copy [23], live migration tools do not guarantee the consistency between the migration source and destination during stop-and-copy. Instead, they fetch all the memory/storage from the migration source after restoring the migrated instance. When applications on the migration destination need a region of memory/storage still absent on the migration destination, live migration tools fetch it from the source. Post-copy has limitations compared to the prior two approaches. First, applications' performance may be limited when live migration tools handle the storms of page faults, or when the network is spotty. Second, the migration source needs to keep the memory/storage of the migrated services until the migration destination has fetched all of it. Application states may be lost if the network or source fails. Therefore, modern data centers typically adopt the combination of pre-copy and stop-and-copy to migrate the memory/storage [1].

Although reducing the blackout time, pre-copy and post-copy incur brownout time, during which services are available, but with performance declines. The major goal of live migration is to minimize the blackout time, and reduce the impacts of brownout.

The networking states mainly consist of two parts, namely, states of communication pipes created by applications (e.g., sockets for TCP/IP), and virtual networks. For the kernel-space network stack, migration of sockets is different in VM, bare-



(a) TCP Migration (VM) (b) TCP Migration (Container) (c) State-of-the-art RDMA Migration (d) MigrRDMA

Figure 1. MigrRDMA and Existing Solutions for TCP/RDMA Migration

metal, and container environments. For VMs, all the socket states are maintained in the network stack of the guest OS. Therefore, as long as the live migration tools restore the guest OS, the sockets become alive again. A similar method is also applied for migrating bare-metal instances (Fig. 1(a)). For containers, as they share the underlying OS, the host network stack needs to provide mechanisms that enable live migration of the sockets. TCP_REPAIR [24] was introduced to the Linux kernel starting from v3.5. It allows live migration tools to extract the socket states (including send and receive queues, packet sequence number, etc.), and to inject these states into the sockets created on the migration destination (Fig. 1(b)). In terms of the userspace TCP/IP network stack (e.g., mTCP [25], Snap [26], etc.), the communication states are maintained in the migrated instance. Thus, the live migration of userspace TCP/IP stack is similar to Fig. 1(a). For all cases, the inflight packets originating from or destined for the migrated services may be lost during live migration. The upper layers of the network stack will treat it as a normal packet loss event and perform the packet loss recovery. In terms of virtual networks, cloud providers need to ensure that packets destined for the migrated services will be forwarded to the migration destinations after live migration. To reduce the blackout time, operators store the new configuration in the network fabric during the pre-copy phase, and activate the new settings during the stop-and-copy step. Before the new configuration fully propagates across the network, operators make the migration source forward the packets it receives during the blackout to the migration destination [1].

B. RDMA Live Migration

RDMA has now been widely adopted in data centers to boost the performance of online services [10], [27]–[31] as it achieves extremely high throughput, ultra-low latency, and high CPU efficiency. Different from TCP which runs in the software, RDMA implements data transport in the hardware. That is the key reason why RDMA achieves those benefits.

However, modern data centers do not support live migration of RDMA. As RDMA offloads data transport to the hardware, RNICs manage most communication states. They are inaccessible from the software. Besides, most commodity RNICs do not provide sufficient network virtualization features inside the hardware to hide the differences between the migration source and destination. Without the virtualization inside the hardware, RDMA live migration requires explicitly notifying the communication partners that the migrated service has now changed to another location. For example, MigrOS [14] introduces a mechanism similar to TCP_REPAIR inside the RNIC to enable extraction and injection of communication states. It extends the RoCEv2 protocol to enable notifying the partners without awareness of the software components. The partner suspends communication with the migration source during live migration, and finally restores communication with the migrated service running on the migration destination[1]. eRDMA [15] offloads the virtualization features (including the network virtualization) to the customized RNICs, and enables the live migration of the hardware states. As the RNICs have the virtualization capabilities, it does not require explicit notification to the partners. Polyakov et.al. [16] proposes a device-assisted live migration for RNIC's VFs, which preserves the resource identifiers exposed to both local applications and remote peers, and quiesces communications of the migrated instances at a packet granularity. They have publicized the solution in Mellanox ConnectX-7 RNICs. All the above solutions require specific hardware supports (Fig. 1(c))[2].

The motivation of this paper is to provide an RDMA live migration solution over commodity RNICs. As Fig. 1(d) shows, we introduce an indirection layer in the kernel, which enables RDMA live migration purely in the software while keeping the RDMA performance benefits and low extra overhead of migration. However, software-based RDMA live migration is challenging in three folds:

***1)* Communication pre-setup during memory pre-copy**. Setting up RDMA connections is slow, taking several milliseconds per connection [17]. It is common in modern data centers for a server to establish hundreds or even thousands of RDMA connections [10], [32]. Thus, establishing RDMA communication during stop-and-copy is unacceptable. RDMA pre-setup during the pre-copy phase is then an attractive solution. However, the challenge lies in registering memory to RNICs, which is a critical step in establishing communications. During the pre-copy phase, memory restoration is complicated, and the structures of the partially restored memory are typically different from the original memory structures of the migrated services. This is because the live migration tool temporarily maps the applications' memory together to a large consecutive

[1]Our work proposes communication suspension in the software to prevent further WRs from being posted on the wire during stop-and-copy (§III-D).

[2]MigrOS was originally designed for containers. This paper is trying to summarize the solutions from a higher view (live migration for both containers and VMs).

region during pre-copy for the convenience of restoring the memory contents. It remaps the memory one by one to the applications' original virtual address after conducting the final iteration of memory restoration. As a result, it is difficult to register memory regions to RNICs using the applications' virtual memory addresses during partial restore (we will interchangeably use "pre-copy" and "partial restore" in the rest of the paper).

2) **Hiding differences between the original and new RDMA communications**. Many RDMA resources and their states are managed by RNICs. These states are exposed directly to the userspace. For example, after establishing communication, applications will get IDs of RDMA communications, and memory access keys, both allocated by RNICs. They are used at each time of data path operation, without the involvement of the OS. Live migration tools cannot control the management of access keys and IDs, therefore unable to keep the states the same between the original and new RDMA communication. Besides, for one-sided operations, applications also need to get the remote memory access keys of the remote side. It is even more challenging to hide the differences in memory access keys from the communication partners of the migrated services.

3) **Consistency of inflight work requests**. Even though live migration tools have frozen the services to be migrated from the software, the RDMA communications inside RNICs are still active. RNICs will still continue processing the inflight work requests (WRs) during the final dumping of the migrated services' states, raising consistency issues of inflight WRs. Live migration tools in the software are unable to get the states of inflight WRs directly from the hardware. Even worse, the migrated service is unaware of the one-sided operations issued by its communication partners. Therefore, it is hard for the software interposing layer to preserve the consistency of inflight operations – making their results consistent as if no migration occurred. It is possible to modify the communication to the initial states to make RNICs discard the inflight WRs, and replay these WRs after migration. However, stopping all the corresponding RDMA communications managed by RNIC is very slow, typically taking milliseconds or even seconds.

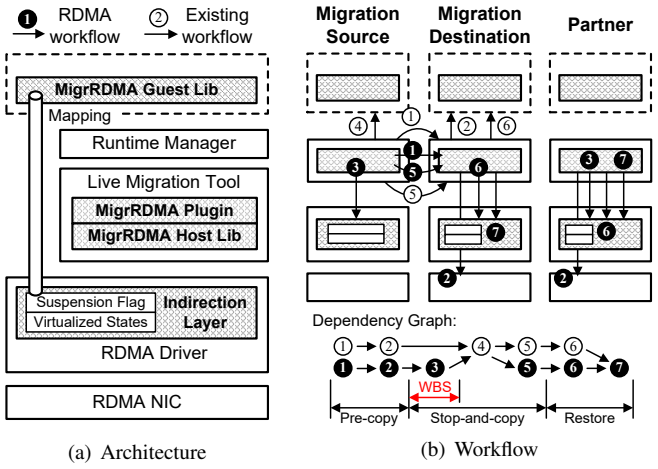
III. Design

This paper proposes `MigrRDMA`. `MigrRDMA` targets both public clouds and virtual private clouds. It mainly provides three features to support RDMA live migration:

- *Pre-setup of RDMA communications during partial restore*. During partial restore, `MigrRDMA` maps the RDMA-related memory structures into the application's virtual memory space, then registers the memory to the RNIC. As such, we can set up all the RDMA communications during the partial restore. `MigrRDMA` provides two different ways to restore the RDMA MRs. For the first method, `MigrRDMA` directly maps the RDMA-related memory structures to the original virtual addresses, and uses the original addresses to register the memory. When remapping the memory at the end of the stop-and-copy, `MigrRDMA` filters out the RDMA-related memory (*Original Address Registration*, OAR). This method requires modification of the memory restoration workflows, which are typically complicated. For the second, `MigrRDMA` maps the RDMA-related memory structures together with the other memories during partial restore, and registers the memory with the temporary virtual addresses. When applications post WRs after restoration, `MigrRDMA` translates the original virtual memory address in the WRs to the address used for MR registration (*Translated Address Registration*, TAR). This method does not need to modify the memory restoration workflows, thus easy to realize.
- *Efficient virtualization of RDMA communications' states*. To hide the differences between the original and new communications, the software indirection layer virtualizes the states exposed to the userspace. These states fall into one of the two categories: they are either used by applications or just used inside the RDMA library. For the former category, `MigrRDMA` maintains the translation table in the kernel, and shares it with the RDMA library. `MigrRDMA` assigns the virtual access keys sequentially, and uses an array to maintain the translation table. For the latter category, `MigrRDMA` hooks inside live migration tools and updates the states after the memory restoration finishes, while keeping transparent to applications.
- *Wait-before-stop to preserve inflight WR consistency*. When the stop-and-copy starts, `MigrRDMA` suspends the RDMA communication to prevent further SEND WRs, and waits for all the inflight WRs to be completed. During this period, applications can still execute the computing tasks. `MigrRDMA` processes the completion notifications on behalf of the application during wait-before-stop. `MigrRDMA` acts as a virtual RNIC to let the application process all the completion notifications previously processed by `MigrRDMA`, until the communication restoration finishes and there are no completion notifications processed by `MigrRDMA`.

A. Overview

Fig. 2 illustrates our design overview. `MigrRDMA` adds three components to the original live migration system (Fig. 2(a)). The indirection layer, residing in the RDMA driver, bookkeeps the minimal states enough to rebuild the RDMA resources. The states can be written only by the RDMA driver, and read only by the live migration tool. Applications have no privilege to access the states. Besides, the indirection layer shares the suspension flag of each QP, and the translation of memory access keys and communication IDs with the RDMA library through memory mapping. The RDMA library has only read access to the shared states. `MigrRDMA` Plugin calls the indirection layer to do RDMA checkpointing and restoration. We realize the `MigrRDMA` Plugin inside the live migration tool. `MigrRDMA` Host Lib and `MigrRDMA` Guest Lib are modified from the existing RDMA library. `MigrRDMA` Host Lib provides extra APIs for live migration tools to restore the RDMA states. `MigrRDMA` Guest Lib suspends communication over QPs, and translates the states based on what is shared by the indirection layer. For simplicity, we use "MigrRDMA Lib" to refer to "`MigrRDMA` Guest Lib"



(a) Architecture (b) Workflow

Figure 2. MigrRDMA Overview

in the rest of our paper, unless stated otherwise. The three components do not rely on the OFED-specific components. Therefore, MigrRDMA can be integrated with both the OFED and Linux upstream drivers.

Fig. 2(b) shows our live migration workflow. We mark the existing memory migration steps and steps to migrate RDMA states with hollow and solid circles, respectively. We also illustrate the dependency graph at the bottom of Fig. 2(b), with the RDMA checkpoint/restore steps and their counterparts for memory checkpoint/restore labeled with the same number. When pre-copy starts, the live migration tool pre-dumps and copies the states of memory to the migration destination (①). The live migration tool starts restoring the memory of the migrated services at the migration destination (②). Meanwhile, MigrRDMA Plugin calls the indirection layer to pre-dump the states of RDMA communication, and copies them to the migration destination (❶). MigrRDMA Plugins on the migration destination and partner call the indirection layer to create or destroy the RDMA communication (❷). The migration source needs to notify the partner through the network (omitted in the figure).

When the final stop-and-copy starts, MigrRDMA Plugin calls the indirection layer to raise the suspension flag (❸). When MigrRDMA Lib detects the suspension flag, it suspends communication over the corresponding QPs, and conducts wait-before-stop (WBS in the figure) to ensure no inflight WRs during the stop-and-copy. To keep RDMA's asynchronous semantics, during communication suspension, MigrRDMA Lib intercepts the WRs to be posted to RNICs, and pretends that they had been posted on the wire. Besides, the migration source needs to notify the partners to suspend communication and conduct wait-before-stop as well (omitted in the figure). The main logic is similar to that on the migration source. The only difference is that on the migration source, we suspend all the RDMA communications created by the applications, while on the partner side, we only suspend the RDMA communication destined for the migration source. The wait-before-stop terminates when all the inflight WRs are completed.

After the wait-before-stop has terminated on both the migration source and the partners, the live migration tool freezes the services to be migrated (④), and dumps and copies the differences of the memory states to the migration destination (⑤). On the migration destination, the live migration tool starts the final iteration of the memory restoration (⑥). Parallel with ⑤ and ⑥, MigrRDMA Plugin dumps and copies the differences of RDMA communication, plus additional information related to RDMA communication state virtualization (❺). It then calls the indirection layer to map the new RDMA resources into the restored processes (❻). After the memory restoration, MigrRDMA Lib on the migration destination and the partner post all the intercepted WRs to RNICs (❼). Finally, the migration source reclaims all the resources of the services that have been migrated (omitted in the figure).

MigrRDMA supports concurrent migration of two services connected with each other. Specifically, the migration destinations of both services establish RDMA communication with each other. The migration destination of the first migrated service may have already set up connections with the service that starts to be migrated later. These connections will be closed.

MigrRDMA supports all the RDMA operations, which include SEND/RECV, READ, WRITE, and ATOMIC. Besides, MigrRDMA has considered all the features of ib_verbs APIs, including completion channels, on-chip memories, and memory windows.

B. RDMA Pre-setup during Partial Restore

Checkpointing the RDMA communication. An RDMA communication contains multiple resources, including protection domains (PDs), memory regions (MRs), completion queues (CQs), queue pairs (QPs), shared receive queues (SRQs), completion channels, on-chip memories, and memory windows, the latter four are optional. The completion channel is an abstraction over the RNIC's hardware mechanism that uses interrupts to notify the WR completion. The on-chip memory allows applications to use the memory of the RNIC for data transmission, reducing the overhead of PCIe transactions [33]. The memory window offers a supplementary layer over the MR to provide finer granularity of access authorization control [34]. Interested readers may refer to specifications [35] and manuals [36] for details.

As most of the RDMA communication states are maintained by the RNIC, it is impossible for us to dump those states. Instead, MigrRDMA maintains a copy of states in the indirection layer. These states are not the full set of states, but the minimal states to rebuild RDMA communications on the migration destination. It is achieved by intercepting all control path calls and maintaining the roadmap of RDMA communication establishment. MigrRDMA not only maintains the information of each individual RDMA resource, but also the dependencies among the RDMA resources. To restore the states on the migration destination, MigrRDMA replays the control path calls. A resource may be created and later destroyed. To avoid unnecessary resource allocation and release during RDMA state restore, MigrRDMA deletes the corresponding resource creation log when the resource is destroyed.

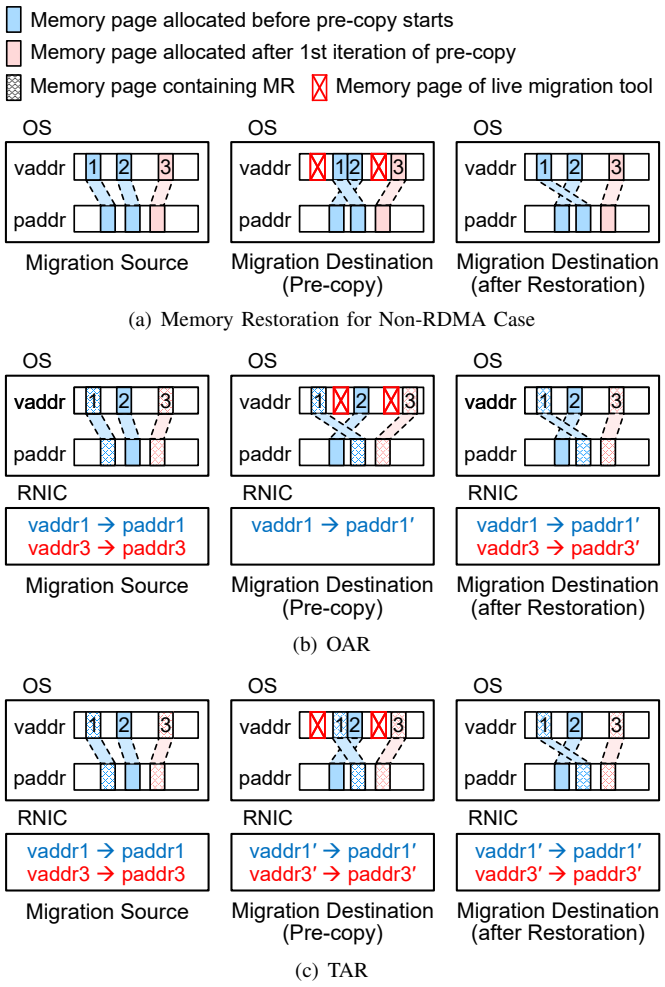
During the partial restore, we not only pre-copy the memory image, but also pre-copy additional RDMA-related informa-

tion for RDMA pre-setup. The live migration tool checkpoints the RDMA communication by interacting with the indirection layer. At the migration destination, `MigrRDMA` establishes RDMA communications equivalent to the original ones based on the checkpointed states of RDMA resources.

Registering MRs during partial restore. Registering MRs is one of the critical steps in establishing communications. Thus, we need to register the MRs during partial restore, and make sure applications can issue data transmission with the original addresses of the MRs. Fig. 3(a) illustrates the original memory restoration for the non-RDMA case. The live migration tool allocates its own memory to store intermediate states, which will be released when memory restoration completes. During pre-copy, the live migration tool temporarily maps the application's memory to a virtual address other than the original one. After finishing restoring, the live migration tool remaps the application's memory back to the original place. It is done by calling `mremap()` system call in Linux, which only alters the virtual memory address and keeps the physical address unchanged. As a result, the application's memory does not conflict with the live migration tool's memory at any time. However, this standard workflow poses a fundamental challenge for RDMA live migration: the MRs need to be registered on the migration destination, but the application's memory resides at a temporary virtual address during pre-copy, making direct MR registration impossible.

`MigrRDMA` provides two methods to address this. The first is OAR (Fig. 3(b)) [37]. The live migration tool identifies the memory structures containing MRs based on the pre-dumped MR's states and the memory table, and maps these memory structures directly to the applications' original virtual addresses. Since the virtual address now matches the original, `MigrRDMA` can register the MRs as normal. The live migration tool checks whether each memory structure contains the MRs, and avoids remapping them to another virtual address. However, OAR requires careful handling of address conflicts, especially when the running service registers new MRs on the migration source during the pre-copy (e.g., Memory Page 3 in Fig. 3(b)). These MRs may conflict with the live migration tool's own memory, even though it is possible to avoid the conflict for the MRs registered before pre-copy by mapping the RDMA-related memory structures before memory restoration starts. We restore the conflicting MRs at the end of stop-and-copy, after the release of the live migration tool's memory. For non-conflicting MRs, we register them during the pre-copy phase. The handling of address conflict requires modification to the existing memory restoration workflows. As memory restoration operates at a low level of the system, such modifications will introduce additional complexity into the development of `MigrRDMA`, and may cause unforeseen failure when restoring the memory in certain cases.

The second method is TAR (Fig. 3(c)). TAR does not distinguish the MRs' memory structures from others, and embraces using the current temporary virtual memory address to register the MRs. To keep transparent to applications, MigrRDMA Lib needs to translate the application's original virtual memory address (denoted as vaddr) to the address used



(a) Memory Restoration for Non-RDMA Case

(b) OAR

(c) TAR

Figure 3. MR Restoration during Partial Restore

for MR registration (translated address, denoted as vaddr′) so that the RNIC can locate the corresponding physical memory based on its on-NIC translation table. Further design details will be presented in §III-C. TAR works seamlessly with the standard memory restoration workflow (Fig. 3(a)), making it significantly easier to realize.

Mapping new RDMA resources into applications. During the stop-and-copy phase, `MigrRDMA` dumps additional RDMA information that is related to RDMA communication virtualization. Such information includes the virtual access keys of MRs and virtual QPNs of QPs. During the final iteration of restore, MigrRDMA Plugin calls the indirection layer to update and virtualize all RDMA-related states to match the new ones.

Establishing new RDMA communication on partners. For connection-oriented communications, the communication partner also needs to establish a new RDMA communication with the migration destination for each QP connected with the migration source. Resetting QPs and re-establishing connections with the QPs on the migration destination would be against the goal of proposing RDMA pre-setup. The reason is that resetting QPs will destroy the communication, and thus can only be done in the stop-and-copy phase. However, resetting QPs is as

Table I
SUMMARY OF STATES MigrRDMA CONSIDERS TO VIRTUALIZE FOR TRANSPARENCY

Exposed to App	Virtualized	Local or Remote	States	Solutions
No	N/A	Local	Queue register, queue metadata, queue pointer, queue content	• Leverage live migration tool to restore the content. • Live migration tool updates the register, metadata, and pointer after memory restoration.
Yes	Yes	Local	Virtual memory address of RDMA MR and on-chip memory	• For MR, leverage OAR/TAR to restore the MR's memory structure (§III-B). • For on-chip memory, remap it to the original virtual address after its allocation on the RNIC of the new location.
Yes	No	Local	Local QPN and local access key	• Manage virtual values and build translation tables.
Yes	No	Remote	Remote QPN and remote access key	• Fetch from the remote side and cache it locally.

slow as setting up new connections, and thus will incur a long blackout time. Our design choice is that the partner establishes new connections with the migration destination, while still continuing data transmission with the migration source over the original QPs. At Step ❻ in Fig. 2(b), the partner switches the communications from the original QPs to the new ones. This requires all the partners to have enough spare QPs to set up new connections. Otherwise, RDMA pre-setup would fail. Nevertheless, modern RNICs have supported more than 10K QPs [38], while applications typically minimize the number of QPs they use to avoid performance scalability issues (e.g., 1,000 QPs [10]). Thus, the problem only happens in extreme cases.

RDMA-based applications usually use one CQ to get completions of multiple QPs. Those QPs may be connected to endpoints other than the migrated instance. Thus, for communication partners, just mapping the new QPs to another CQ would break the transparency. Instead, we let the old and new QPs share the same CQ on the partner. Other resources (e.g., PDs, SRQs if any) can all be reused on the partner. Thus, different from the migration destination that needs to create all the RDMA resources, the partner only needs to establish new QPs.

As the partner may have established connections with the servers other than the migration source, the partner needs to know for which QP it should establish a new one with the migration destination. To achieve this, MigrRDMA lets the live migration tool on the migration source notify all the partners. The notification message contains the network address of the migration destination and a list of physical QPNs of each partner (if any). For each physical QPN the partner receives, the partner establishes a new QP and communicates with the migration destination to exchange the new physical QPNs. To figure out which partner MigrRDMA needs to notify, and the list of physical QPNs to be sent to each partner, we add the fields of the destination physical QPN and the destination network address to the metadata of the connection-oriented QP. Finally, the partner translates the original physical QPN to the virtual QPN (the related data structure will be introduced in §III-C), and maps the virtual QPN to the new QP.

C. Hiding Differences between Source and Destination

The newly established RDMA communications are different from the original ones in many aspects. MigrRDMA needs to provide a virtualization layer to ensure transparency for applications. We summarize all RDMA states that are different between the migration source and destination. These states fall into one of the four types according to their characteristics. We take different actions for each type to achieve efficient virtualization. Table I summarizes the states and their characteristics. We will discuss the detailed operations of hiding the differences one by one.

Local states that are hidden from applications. These states are managed in the RDMA library or driver. They include the registers, metadata, head and tail pointers, and the contents of the queues (i.e., QPs, CQs, and SRQs). For the register, metadata, and pointers, MigrRDMA only needs to update them to the new values. No virtualization is required for this type. For queues, MigrRDMA leverages the live migration tools to restore the queue contents.

Local states that have been virtualized. Some states are exposed to applications and have been virtualized for applications. The virtual address of the MR and on-chip memory belongs to this type. To hide the difference of physical addresses, MigrRDMA needs to guarantee that all the memory tables are correct. MigrRDMA provides OAR/TAR to restore the MRs (§III-B) so that applications can use the MR's original virtual addresses. In terms of the on-chip memory, the RDMA driver allocates the NIC's memory with the size specified by the application, then maps the NIC's memory into the application's virtual address space. Therefore, when restoring the on-chip memory on the new location, the live migration tool needs to allocate it with the same size, and remaps it to the original virtual memory address.

Local states that have not been virtualized. The third type is the local states that are used by applications, and the values are not virtualized. The local QP Number (QPN) and the local access key (*lkey*) of each MR belong to this category. MigrRDMA maintains translation tables from virtual to physical, or vice versa. Special considerations need to be taken for translation efficiency.

For local QPN, the translation happens only in the physical-to-virtual path. The indirection layer in the RDMA driver maintains the QPN translation table as an array. The index denotes the physical QPN, and the corresponding value is the virtual QPN. As the physical QPN is unique within each RNIC, each physical QPN has only one virtual QPN. The specification uses 24 bits to represent a QPN [35]. Thus, the array has 2^{24} entries, each taking 4 bytes. This array resides in the kernel and is shared by the MigrRDMA Lib of all the processes. Thus, the total memory consumption

only depends on the number of RNICs the applications use (64 MB per RNIC). When the application creates a new QP, the RNIC returns the local QPN in the QP context. `MigrRDMA` just sets the virtual QPN the same as the physical value. When the application processes the CQ entry, it needs to use the local QPN inside the CQ entry to identify the QP from which the current completed request is. `MigrRDMA` Lib translates the local QPN from physical to virtual when the application processes the CQ entry. During RDMA pre-setup, the translation table is updated when the new physical QPN is assigned.

For *lkey*s, the translation happens from virtual to physical ones when applications post requests on QPs. To reduce table lookup overhead, `MigrRDMA` manages the virtual values in a dense way – the virtual values are assigned one by one. With this design, the indirection layer maintains the translation table as an array. The corresponding index is the virtual *lkey* exposed to the application. To avoid security issues raised by this simple key allocation method, `MigrRDMA` maintains the table at a per-process granularity. The process ID combined with the virtualized *lkey* then becomes the key of translation table entries. An attacker is unable to forge a virtualized access key and attack an RDMA-based service under such a design. `MigrRDMA` dynamically allocates/deallocates the memory pages for the access key translation table – when the table size exceeds the current capacity, `MigrRDMA` allocates a new page to store the new translation entry, and reclaims the empty pages upon the deletion of the entries. Therefore, the memory usage scales linearly with the number of MRs in a stepwise and quantized manner. For OAR scheme, the value is the physical *lkey*. Each entry takes 4 bytes. For TAR scheme, the local virtual memory address is translated at the time of *lkey* translation. The value is the tuple of physical *lkey*, original ($vaddr_0$), and translated starting address ($vaddr'_0$) of the MR. Thus, each entry takes 20 bytes. Suppose the application specifies $vaddr$ in the WR. MigrRDMA Lib calculates the offset as $(vaddr - vaddr_0)$, then adds the offset to $vaddr'_0$, and finally populates it in the WR. Admittedly, applications may register plenty of MRs, and thereby the array can be very large. Nevertheless, operators typically minimize the total number of the MRs (e.g., below one hundred [29], [39]–[41]) for performance concerns. In this case, the *lkey* translation does not incur massive overhead.

Remote states that have not been virtualized. The fourth type is the states that have not been virtualized like the previous type, but the values are maintained in applications running on remote nodes. Remote QPNs and remote access keys (*rkey*s, including both the MRs' and memory windows' *rkeys*) belong to this type. Applications usually use out-of-band channels to exchange remote QPNs and *rkey*s [42], [43]. The RDMA library or the RDMA driver is unaware of such an exchange. There are three cases that require `MigrRDMA` to translate remote QPNs or *rkeys*. 1) For remote QPN of connection-oriented communications, the translations are only required at connection setup, as the RNIC needs to use the physical QPN to connect with the other. The remote QPN will not be needed during the communication. 2) For remote QPN of datagram communications, translations are needed for each request. 3) For *rkey* of one-sided operations (i.e., READ, WRITE, ATOMIC), the translations are required for each request. For the latter two cases, MigrRDMA Lib caches the mapping of *rkey*s and remote QPNs to translate them locally. At first, the whole cache is invalidated. For the first time of translation from a particular *rkey* or remote QPN, MigrRDMA Lib fetches the corresponding physical one from the remote side. Then, the subsequent translation from those *rkey*s and remote QPNs is only done locally, without extra communication with a remote server. The overhead mainly comes from the remote fetching upon the first translation, which is amortized over all the subsequent translations. During live migration, the migration source invalidates all its communication partners' caches that store the *rkey*s and QPNs of the migrated service. After restoration, the partners need to fetch new physical *rkey*s and remote QPNs from the migration destination. For TAR scheme, the remote virtual memory address is translated at the time of *rkey* translation in a similar way to the local virtual memory address.

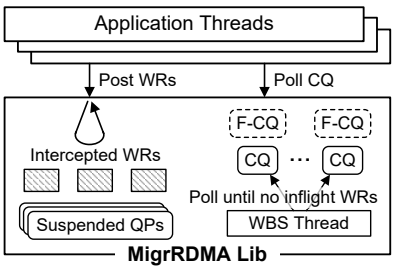


Figure 4. Illustration of Handling WR Consistency

D. Inflight Request Consistency

The design choice of `MigrRDMA` to handle inflight request consistency is wait-before-stop. When stop-and-copy starts, MigrRDMA Plugin calls the indirection layer to raise the suspension flag, and notifies the MigrRDMA Lib to conduct wait-before-stop. When all the inflight WRs are completed, the live migration tool proceeds with the final stop-and-copy. We have considered other methods such as drop-and-replay. However, we do not adopt it for two reasons: 1) replaying causes a similar time to wait-before-stop as their performance bottleneck is network throughput, and 2) discarding all the WRs on the QPs is time-consuming.

A critical issue to consider is how to conduct wait-before-stop while ensuring transparency to applications. The RDMA data path bypasses the kernel. `MigrRDMA` Lib is then the optimal point for wait-before-stop execution. `MigrRDMA` Lib is loaded within the process space of each application. We need to find a possible point to conduct wait-before-stop in the `MigrRDMA` Lib without affecting the applications (e.g., they will not stall on the communication suspension).

Core mechanism. Fig. 4 shows the key design of handling inflight WR consistency. MigrRDMA Lib spawns a dedicated wait-before-stop thread (WBS thread) when being loaded into each process space for the first time. This thread listens to the indirection layer, and keeps suspended until the indirection layer sends a signal. Thus, the WBS thread does not contend for CPU resources with the application's threads. When the

indirection layer sends the pause signal, this thread takes over the RDMA communication and performs two tasks. First, it suspends RDMA communication to prevent further WRs. Second, it proactively polls all relevant CQs to track the completion of inflight WRs. `MigrRDMA` maintains a per-CQ fake CQ. On the one hand, the fake CQ prevents applications from stalling on the suspended communication. The WBS thread places the CQ entries into the fake CQ. If an application polls the CQ during communication suspension, MigrRDMA Lib redirects it to the fake CQ. On the other hand, the fake CQ facilitates consistency in handling the CQ entries. The fake CQ stores the CQ entries not consumed by applications. After restoration, MigrRDMA Lib redirects the polling of the CQs to their fake CQs until the fake CQ becomes empty.

Communication suspension. During communication suspension, the WBS thread raises the suspension flags of the QPs. For the migrated service, all the QPs should be suspended. For the partners, only the QPs connected to the migrated service need to be suspended. When applications post WRs, MigrRDMA Lib will intercept the WRs and return without blocking if detecting the suspension flag of the QP. The intercepted WRs are buffered in the memory. With this design, applications do not stall on the suspended QPs. They only perceive that they get the CQ entries slower than before, but they can deal with computation tasks as normal. Both sides of the RDMA communication will issue the intercepted WRs on the RNICs when the live migration tool restores the RDMA communication.

Wait-before-stop. After the communication suspension, the WBS thread keeps polling the CQs until all the inflight WRs are completed. During this phase, applications can process the CQ entries as normal. The proactive polling by the WBS thread is critical to achieve deterministic blackout time. If we only relied on the application to consume the CQ entries, the wait-before-stop would take a long time if the application were busy doing something other than CQ polling (§V-B3).

The key to wait-before-stop is to identify whether there are inflight WRs or not to determine the termination condition. In the regular workflow of the CQ entry processing, for each CQ entry, the RDMA library identifies the QP the completed WR is from, and then updates the tail pointer of the SQ/RQ (send queue/receive queue). The window capped by the head and tail pointers of the SQ/RQ is exactly the inflight WRs. Thus, for the SQs, there are no inflight SEND WRs if and only if tail pointers of all the SQs reach the head pointers. For the RQs, the termination condition is different. For two-sided verbs, the receiver needs to post RECV WRs before the sender posts SEND WRs. The WRs in the receive queue of one side are more than those in the send queue of the other side. Therefore, there are no inflight RECV WRs when and only when the number of two-sided verbs the peer has posted (denoted as n_sent) equals the number of completed RECV WRs (denoted as n_recv). We add fields in the QP metadata to maintain the number of posted two-sided verbs and completed RECV WRs for each QP since its creation. If the number of posted two-sided verbs is non-zero, the WBS thread sends the n_sent to the other side. If receiving n_sent from the other side, the WBS thread compares n_sent and n_recv. For the inflight RECV WRs without the other side posting corresponding SEND operations, `MigrRDMA` will replay the RECV WRs on the migration destination and partners when restoring the communication.

Consistency of CQ entries not consumed by applications. For the migrated services, `MigrRDMA` creates new RDMA communications on the migration destination. Thus, the original CQ entries are lost. However, applications may still not consume all the CQ entries at the time of live migration. We need a mechanism to preserve the consistency of the CQ entries. To do this, `MigrRDMA` migrates the fake CQ containing the unprocessed CQ entries together with the regular memory contents. After restoration, MigrRDMA Lib first polls the fake CQ. When the fake CQ becomes empty, it polls the real CQ as normal and destroys the fake CQ. MigrRDMA Lib needs to translate the physical QPNs in the CQ entries from both the fake and real CQs back to the corresponding virtual QPNs. To do this, when wait-before-stop terminates, MigrRDMA Lib maintains a temporary translation table from the original physical QPN to virtual QPN for all the QPs. After restoration, if the CQ entry comes from a fake CQ, MigrRDMA Lib translates the QPN based on the temporary translation table. When finding a CQ entry from a real CQ, MigrRDMA Lib deletes the corresponding table entry (because there will no longer be CQ entries for the WRs of the old QPs). Then, it translates the QPN based on the QPN translation table managed by the indirection layer (mentioned in §III-C).

On the partners, the new QP share the same CQ with the old one. After the restoration of communication, the CQ contains the CQ entries for both the old QP and the new one. The QPN translation table of the indirection layer has recorded the entries for both the old and new QPs (i.e., old pQPN to vQPN when the application creates the QP, new pQPN to the same vQPN when the partner creates the new QP during partial restore). MigrRDMA Lib can naturally translate the QPNs recorded in all the CQ entries. When all completions belonging to old QPs are consumed, the old QPs and related QPN translation table entries are destroyed.

Consistency of CQ events. Handling CQ consistency in interrupt mode (using CQ events) is different from polling mode. As applications will process CQ entries immediately when notified by CQ events, there is no need to use the wait-before-stop thread to poll the CQ. What `MigrRDMA` does is to make sure all CQ entries of a CQ that uses events are processed before live migration. Thus, `MigrRDMA` tracks the number of unfinished CQ events in `MigrRDMA` Guest Lib. The number is increased when the application waits for an event, and decreased when an event is successfully handled. The absence of any unfinished CQ events (i.e., the number equals zero) is another necessary condition for the termination of wait-before-stop.

Handling buggy network situations. The live migration may happens in a spotty network. In this case, simply waiting for all the inflight WRs delivered would cause an unexpectedly long time during wait-before-stop. To handle this case, `MigrRDMA` sets an upper bound on the time elapsed for wait-before-stop. If the time expires and there are still

WRs not completed, MigrRDMA directly starts stop-and-copy. After finishing restoring the applications, the migration destination and partners first replay the WRs previously posted but not completed, before replaying the WRs intercepted by MigrRDMA Lib.

IV. Implementation

Our implementation is targeted for container live migration. Container live migration involves the following components: a CLI tool to manage the container runtime (e.g., runc [19]), and a live migration tool (e.g., CRIU [18]). The cloud manager calls runc to checkpoint and restore. Runc in turn calls CRIU to perform the corresponding commands.

The existing CRIU and runc do not offer the full ability for live migration. First, CRIU and runc do not support partial restore during the memory pre-copy. Second, runc does not support migrating the processes other than the initial one. For the former, we add PartialRestore command in runc, which calls the restore command of CRIU. We also break the restore procedure of CRIU into two parts. The first part is partial restore, and the second part is full restore. Before the full restore starts, we add a logic inside CRIU to wait for the stop-and-copy signal. We extend runc with FullRestore command to signal CRIU through the UNIX socket. For the latter, we write a script to get the root process IDs of the initial and all the non-initial processes by reading the files maintained by Docker. Then, we start a CRIU for each root process ID to do checkpointing and restoration. The non-initial processes are restored through our extended Exec command.

To add support for RDMA live migration, we add CheckpointRDMA command in runc. It dumps the container images containing the RDMA-related information. If a previous dump file exists when this function is called, it generates only the difference instead of the full set. RDMA pre-setup can only map memory at the beginning of restoration on the target. For the updated RDMA memory during pre-setup, its recovery is delayed to the end of full restore. Thus, we only need to dump RDMA states twice, one at the beginning of pre-copy, the other at the stop-and-copy. However, we still support multiple rounds of memory pre-copy. During partial restore, CRIU maps the RDMA-related memory structures to the original virtual memory addresses before the memory restoration starts, then establishes all the new RDMA communications. All the command extended in runc are listed in Table II.

To evaluate the benefits of RDMA pre-setup, we implement another RDMA live migration workflow without communication pre-setup for comparison. For this case, we only do one dumping during stop-and-copy. The RDMA dumping logic is the same. In terms of restoring, we restore the RDMA after all the memory are restored.

We implement a prototype of MigrRDMA based on Mellanox OFED 5.4 driver, including ∼4,000 lines of C code (LoC) added in the RDMA kernel-space driver, and ∼13,000 LoC in the RDMA library. The kernel-space driver maintains all the required information to create equivalent RDMA communication. The MigrRDMA Lib in the host offers the extended APIs to CRIU (Table III). The MigrRDMA Lib in the container translates QPNs and access keys based on the table shared by the RDMA driver, and handles inflight request consistency. We add around 5,700 LoC to CRIU, and around 1,200 LoC to runc. We do not modify any codes of Docker.

Table II
Description of Extended Commands in Runc

Command	Description
CheckpointRDMA	Dump the container images containing the difference of memory and that of RDMA-related information.
PartialRestore	Execute the existing restore command of CRIU. The restore procedure of CRIU is broken into partial restore and full restore.
FullRestore	Signal CRIU to start full restore.
Exec	Extend the existing Exec command to support live migration feature.

Table III
MigrRDMA's APIs

Existing RDMA APIs	APIs for CRIU
ibv_open_device()	ibv_restore_context()
ibv_alloc_pd()	ibv_restore_pd()
ibv_reg_mr()	(Does not need extra API)
ibv_create_cq()	ibv_restore_cq()
ibv_create_qp()	ibv_restore_qp()
ibv_modify_qp()	(Does not need extra API)

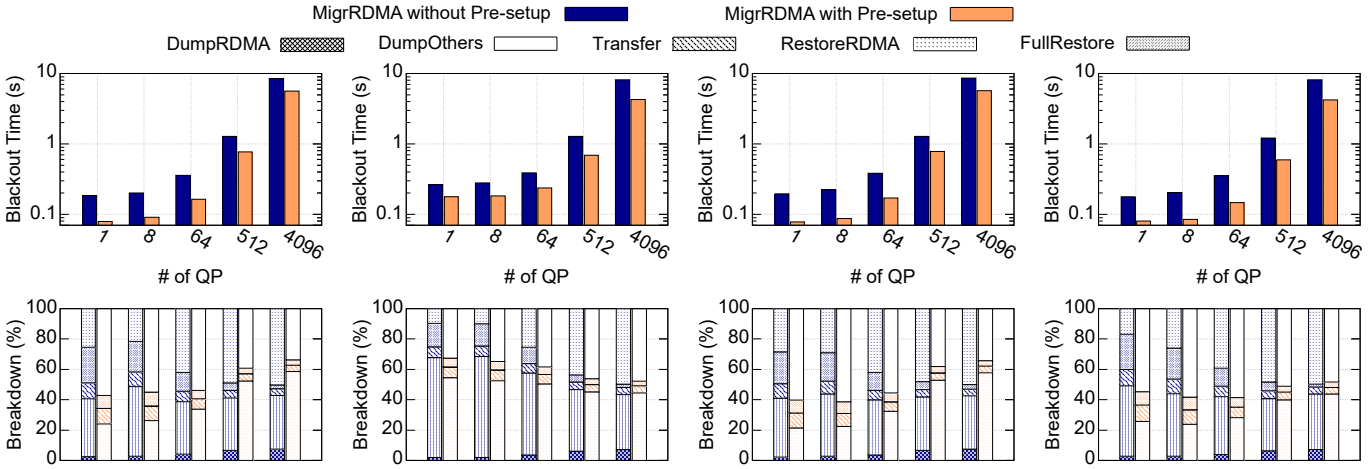
V. Evaluation

This section conducts evaluations on our MigrRDMA prototype. The main conclusions from the evaluation results are:

- The restoration of RDMA communication contributes significantly to the migration time. Adopting RDMA pre-setup reduces the blackout time by up to 58%.
- Proactive CQ polling during wait-before-stop is necessary. It can reduce the communication blackout time by 4 orders of magnitude under the communication-sparse workload.
- RDMA live migration incurs around 9% end-to-end performance gap under large scale. The overhead of wait-before-stop is relatively small compared to the total communication blackout time. The brownout has little effect on the performance.
- The virtualization layer of MigrRDMA does not introduce massive overhead in the data path, only 2% $\sim$ 9% extra CPU overhead for each operation, resulting in marginal overhead in the end-to-end latency.

A. Evaluation Setup

Our testbed consists of six servers, representing a migration source, a migration destination, and four communication partners, respectively. Each server has an AMD EPYC 7452 32-core processor, 256 GB memory, and a Mellanox ConnectX-5 100 Gbps RNIC. The RNICs are connected through an Arista 7260CX3-64 switch.



(a) Migrated Service is Posting Out-bound WRITE

(b) Migrated Service is Serving In-bound WRITE

(c) Migrated Service is Posting SEND

(d) Migrated Service is Posting RECV

Figure 5. Breakdown of MigrRDMA's Blackout Time

Our evaluation of MigrRDMA mainly consists of five parts. Firstly, we validate the critical features of MigrRDMA, including the correctness of RDMA live migration, benefits of RDMA pre-setup, and the necessity of proactive CQ polling. Secondly, we assess the overhead of RDMA live migration by analyzing the impacts on end-to-end performance and the overhead of wait-before-stop. Thirdly, we evaluate the overhead of MigrRDMA's virtualization by measuring the end-to-end latency and the CPU cycles incurred in each data path operation. Then, we analyze the primary differences between our work and the state-of-the-art RDMA live migration scheme for containers, MigrOS [14], and estimate the extra blackout time of MigrOS compared to MigrRDMA. Finally, we use MigrRDMA to migrate an RDMA-based Hadoop [44]. We use the binary code of RDMA-based Hadoop without any modification.

To evaluate MigrRDMA, we extend perftest [45] for correctness checking, one-to-many communication pattern generation, and measurement of CPU cycles taken by data path operations. It does not mean that MigrRDMA requires modification of applications. The measured migration time is sensitive to the resource contention. Therefore, in the one-to-many communication, we scale the partners across the four servers, each partner using one dedicated CPU core. Our testbed supports 128 partners.

B. Feature Validation

1) Correctness of RDMA live migration: We need to confirm that after migration, applications can get the completion of all the WRs in order, without duplication, loss, and content corruption. RDMA has provided a WR ID field in each WR that is populated by applications. Upon WR completion, the RNIC fills the WR ID in corresponding CQ entry [35]. This allows applications to check whether each individual WR is completed or not. Therefore, we extend perftest to populate the WR ID with a sequence number we define for each QP. By checking the order of the WR ID returned from the CQ entries, we can identify whether all the WRs are completed in order, without duplication and loss. In terms of the content of the CQ entry, we check if any field conflicts with the local inferred value. During the evaluation, there is no error information that we added printed out, indicating that we have guaranteed the consistency of inflight WRs.

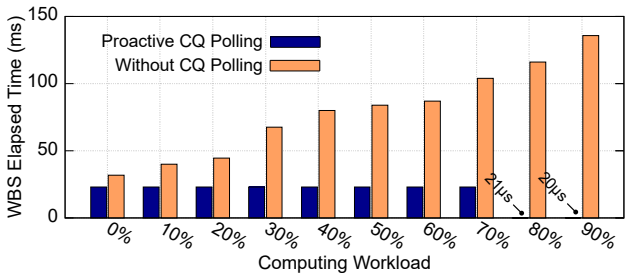
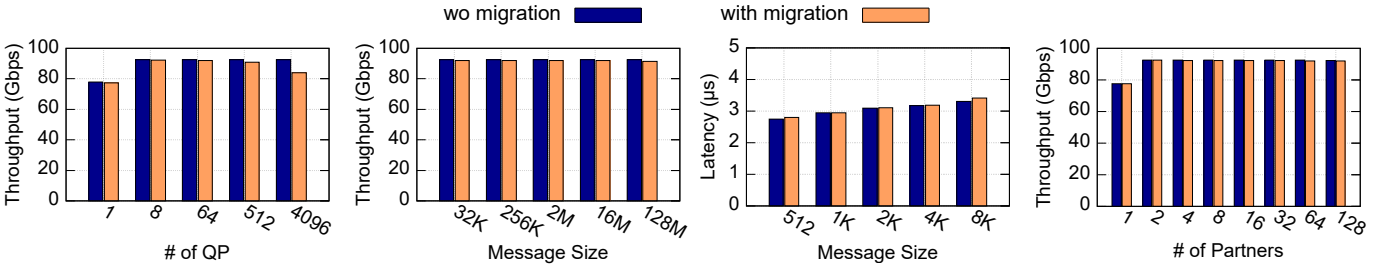


Figure 6. Necessity of Proactive CQ Polling

2) Benefits of RDMA pre-setup: We implement MigrRDMA for the case without and with RDMA pre-setup, and break down their blackout time. The blackout time consists of five parts (if any): DumpRDMA, DumpOthers, Transfer, RestoreRDMA, and FullRestore. For the case with RDMA pre-setup, the blackout time only includes DumpOthers, Transfer, and FullRestore. In terms of the case without RDMA pre-setup, all the five parts are included in the blackout time.

Fig. 5 shows the results of the breakdown. As the number of QPs increases, the elapsed time of RestoreRDMA grows significantly (when the number of QPs reaches 4,096, RestoreRDMA takes nearly 50% of the blackout time in the case without pre-setup). Adopting RDMA pre-setup can significantly reduce the blackout time. As the number of QPs increases, the time of DumpOthers increases significantly even for MigrRDMA with pre-setup. This is due to the inefficient CRIU implementation for large and complicated memory structures which has been reported before [14]. For MigrRDMA with pre-setup, the elapsed time of DumpOthers in the case of migrating the sender (Fig. 5(a) and 5(c)) increases more rapidly than in the case of migrating the receiver (Fig. 5(b) and 5(d)). We speculate that it is because the memory structure of the sender is more complicated than that of the receiver.



(a) Throughput against # of QPs (4 KB Message)

(b) Throughput against Message Sizes (1 QP)

(c) Latency against Message Sizes (1 QP)

(d) Throughput against # of Partners (1 QP per Partner, 4 KB Message)

Figure 7. Difference in Throughput and Latency with and without Live Migration (Queue Depth: 64)

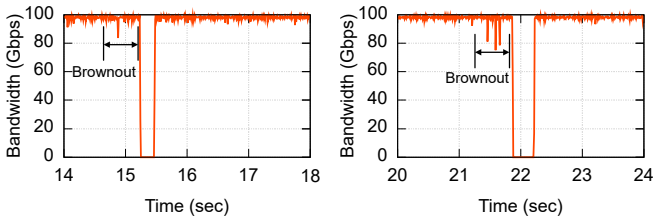
3) Necessity of proactive CQ polling: To evaluate the necessity, we measure the elapsed time of wait-before-stop under the case with and without proactive CQ polling. The scenario we consider is that applications do both communication and computing tasks on a single CPU core. To emulate that, we leverage `cgroups` to throttle the CPU utilization of `perftest`, assuming that the rest of the CPU is for the computing task. It is guaranteed that the WBS thread runs on another core, and it is not throttled. Fig. 6 shows the evaluation result under the settings of 4-MB messages and 64 queue depth. With proactive CQ polling during wait-before-stop, the elapsed time keeps ∼23 ms, except for the very high computing workload. When the computing workload reaches 80%, there are no inflight WRs when the wait-before-stop starts, and the WBS thread does not do any CQ polling, resulting in µs-scale elapsed time. In contrast, without CQ polling, the overhead of wait-before-stop grows with the computing workload. This is because when communication takes fewer CPU resources, the application consumes CQ entries slower, causing a longer time to wait for all the inflight WRs' completion.

C. Overhead of RDMA Live Migration

1) Impacts on end-to-end performance: We evaluate the performance impact from two aspects: i) the difference in throughput and latency with and without live migration, and ii) fluctuation of throughput during live migration.

For the former, we run `perftest` for 1 min, and migrate the sender side during the runtime. The performance gap caused by live migration mainly depends on the blackout time, which is in turn influenced by the following three factors: a) the number of QP, b) the message size of inflight WRs, and c) the number of partners. `Perftest` does not support latency test in multi-QP settings. Thus, we only show the latency result under different message sizes. We omit the evaluation of migrating the receiver side because the results are similar. As Fig. 7 shows, `MigrRDMA` causes little difference in throughput and latency when performing live migration. In terms of throughput, `MigrRDMA` induces ∼9% gap under the large scale (i.e., 4,096 QPs). For latency, `MigrRDMA` only introduces sub-microsecond overhead.

For the latter, `perftest` does not support fine-grained throughput measurement. Thus, we leverage counters provided



(a) Migrate Sender Side, Measure Receiver Side

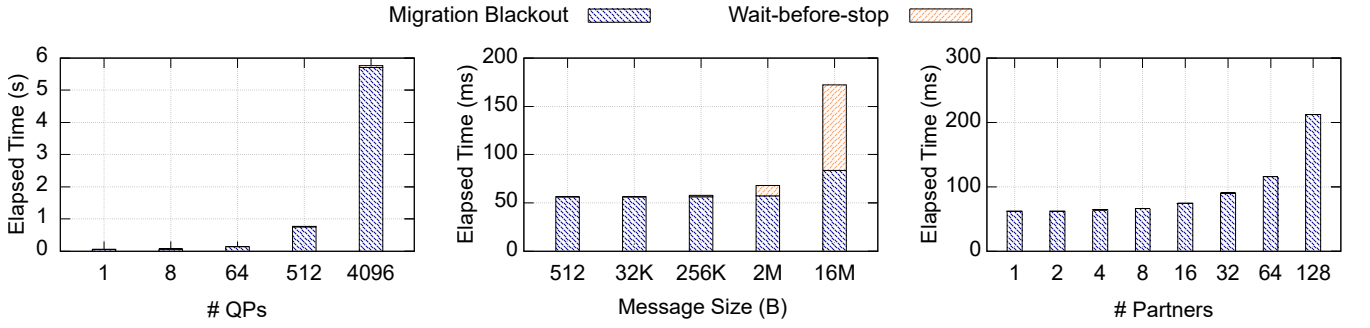
(b) Migrate Receiver Side, Measure Sender Side

Figure 8. Throughput Fluctuation during Live Migration

by Mellanox RNICs [46] that indicate how many bytes have been sent or received. By sampling the counters and the current timestamp repeatedly, we can obtain the real-time throughput fluctuation. The log we print shows 5-ms time granularity, which is enough for the evaluation. In this evaluation, we migrate a container running `perftest` that transmits 2-MB messages using one-sided verbs through 16 QPs. At the same time, we sample the throughput fluctuation on the partner side. The cases of migrating the sender and receiver sides are both evaluated. Fig. 8 shows the real-time throughput of the partner side. In both scenarios, the partner side perceives slight performance implications during the brownout time. This phenomenon is due to the resource contention on RNIC and was first reported by Kong et al. [47]. Although the brownout time may be long since establishing all the RDMA communication is slow, the brownout has little impact on the performance. When wait-before-stop starts, the partner does not receive any messages until the communication is restored. The blackout time lasts about 150 ms.

In summary, `MigrRDMA` causes little impact on applications' end-to-end performance when doing live migration.

2) Overhead of wait-before-stop: In this set of experiments, we assess how much overhead wait-before-stop incurs, taking the case of migrating the sender of two-sided verbs as an example. We change the three factors affecting the elapsed time of wait-before-stop (Fig. 9). Besides, we compare the elapsed time of wait-before-stop to the migration blackout time under the varying three factors, separately. As Fig. 9 shows, in most cases, wait-before-stop contributes little to the communication blackout time. In Fig. 9(b), the elapsed time of wait-before-stop dominates the communication blackout when

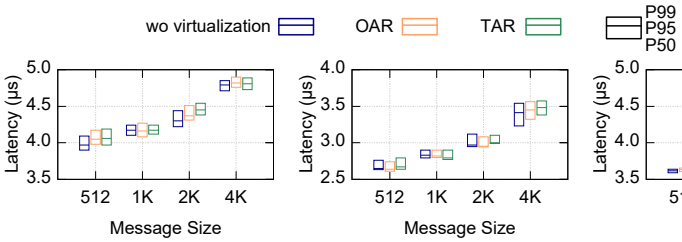


(a) Wait-before-stop with Different # QPs (4 KB Message, 1 Partner)

(b) Wait-before-stop with Varying MsgSize (1 QP, 1 Partner)

(c) Wait-before-stop with Different # Partners (1 QP for each partner, 4 KB Message)

Figure 9. Overhead of Wait-before-stop (Queue Depth: 64)



(a) SEND (b) WRITE (c) READ

Figure 10. End-to-end Latency with and without Virtualization

Table IV
CPU CYCLES OF EACH OPERATION

Op	CPU Cycles			Overhead	
	w/o virt	OAR	TAR	OAR	TAR
SEND	112.2	119.5	120.3	6.5%	7.2%
RECV	63.5	68.8	69.2	8.3%	9.0%
WRITE	108.2	112.0	113.6	3.5%	5.0%
READ	108.4	110.9	111.4	2.3%	2.8%

the message size reaches 16 MB, as the network becomes the bottleneck in this case. In terms of wait-before-stop under different numbers of partners, we extend `perftest` to support one-to-many communication pattern. The migrated container starts one `perftest` and creates n QPs, while each of the n partners starts one `perftest`, creates one QP, and connects it to the migrated container. Before live migration starts, we close the n sockets that negotiate the QPNs, as the overhead of migrating the sockets is non-trivial. By comparing the results in Fig. 9(c) with those in Fig. 5(c), we find that the blackout time under the multi-partner case is almost equivalent to that under the single-partner case with the same number of QPs, indicating that synchronization and notification have negligible overhead without network loss or congestion.

D. Overhead of Virtualization

1) End-to-end latency: First, we evaluate the end-to-end latency without virtualization, and with virtualization under the scheme of OAR and TAR. We perform latency tests with `perftest` for SEND, WRITE, and READ operations, varying the message size from 512 B to 4 KB. We set the queue depth as 1 and the MTU to be greater than the message size (i.e., 4,096) to minimize random errors. Each test runs 10^6 iterations. Fig. 10 shows the median, P95, and P99 latency, which indicate that the virtualization layer under both OAR and TAR only introduces marginal overhead in the end-to-end latency.

2) CPU cycles: Next, we measure and compare the CPU cycles of SEND, RECV, WRITE, and READ of a single RC QP in the case with and without virtualization. We slightly modify the `perftest` to sample the CPU cycles of each invocation of these operations. The measurements are conducted under the message size of 64 B for 5 seconds. As the CPU is the bottleneck under such a message size, the uncontrollable CPU cycles for busy polling are excluded. The results in Table IV demonstrate that both OAR and TAR incur negligible overhead in the data path, adding only 2.5 $\sim$ 8.1 extra cycles in these operations, 2% $\sim$ 9% additional overheads in the data path. Assuming typical cloud servers with 2 $\sim$ 3 GHz CPU clock frequency, the per-WR overhead is about 8.3×10^{-10} to 4.1×10^{-9} CPU cores. Supporting 100 million RDMA operations per second (such throughput is just an extreme case) only requires 0.08 to 0.41 CPU cores.

E. Comparison with MigrOS

MigrOS [14] needs to modify the RNIC. The authors do not provide a hardware prototype, but provide implementation over SoftRoCE [48] for feature verification. As SoftRoCE has severe performance declines, the current MigrOS prototype is not a good choice for performance comparison. Thus, we instead examine the key differences between `MigrRDMA` and MigrOS, and provide an analysis. For fair comparison, we suppose MigrOS had achieved memory pre-copy, whose performance is dominated by memory allocation and access characteristics. As such, the major differences between MigrOS and `MigrRDMA` lie in stop-and-copy.

Stop-and-copy consists of three steps: 1) waiting for the RDMA communication states to be safe for live migration, 2) transferring all the states to the new location and restoring at the final iteration, and 3) replaying what applications have previously posted but still not issued on the wire. Only the second step is the service blackout time, and all of the three steps constitute the communication blackout time. For the waiting step, MigrOS chooses to stop the communications on

Table V
ESTIMATION OF STOPPING QPS AND INJECTING STATES

# QP	Stopping QPs (ms)	State Injection (µs)
1	1.48	0.16
8	2.52	0.25
64	3.60	0.31
512	24.04	0.50
4,096	192.25	2.81

the hardware and to let RDMA packets naturally propagate the information to communication partners. During the replaying step, MigrOS transmits all non-acknowledged and pending packets. As the performance bottleneck is the amount of data that needs to be transmitted, `MigrRDMA` and MigrOS have almost the same performance for the total time of these two steps, except for stopping QPs. In terms of the second step, MigrOS needs more time to extract and inject the states from and to the RNIC, respectively. In contrast, `MigrRDMA` does not need to get/set the detailed states from/to the RNIC. As the metadata is stored in memory, `MigrRDMA` can leverage the existing memory live migration scheme to migrate the RDMA states. Based on the above theoretical analysis, the extra blackout time of MigrOS vs. `MigrRDMA` includes stopping QPs on the hardware, and state extraction/injection.

To provide quantitative analysis, we write an RDMA-based application to measure the elapsed time of concurrently resetting QPs and estimate that of state injection. To emulate state injection, we post a SEND operation on a UC QP with the message size the same as the total size of the communication state[3], and calculate the elapsed time between posting SEND and getting the corresponding CQ entry. As Table V shows, the overhead of stopping QPs on the hardware is massive – 100ms-scale extra blackout time when the number of QPs is 4,096. In conclusion, we estimate that the blackout time of MigrOS is longer than that of `MigrRDMA`.

F. Migration of Real-world Applications

Finally, we use `MigrRDMA` to migrate a real-world RDMA-based application. The application we choose is Hadoop File System (HDFS) [49], [50]. We use the binary of the RDMA-based Hadoop [44] without any modification.

In the evaluation, we start a master and two slave containers on two servers, respectively. We submit a task on the master node, and the master node distributes tasks to one of the servers. During the runtime of the task, suppose the operator needs to restart the server running the slave container for maintenance. With `MigrRDMA`, operators can migrate the slave container to another server. However, without RDMA live migration scheme, operators leverage the Hadoop's native failover mechanism for the application's reliability. Specifically, the master node detects the loss of the slave node. It then re-distributes the tasks to a backup container on another server, and recovers their runtime on the migration destination based on the log of the tasks. In this evaluation, we compare the application-perceived throughput (Tput) and job completion time (JCT) of baseline, `MigrRDMA`, and failover.

[3]Suppose the size of each QP's state is 375 B [38].

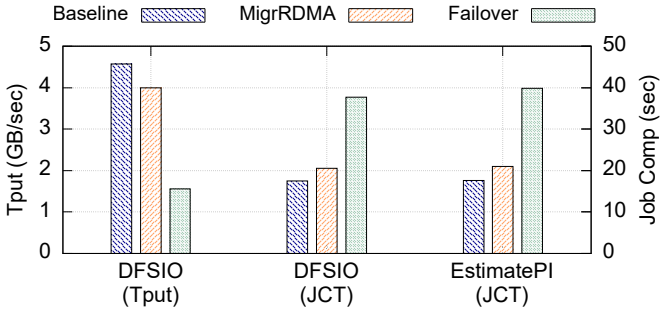


Figure 11. Migration of RDMA-based Hadoop

Fig. 11 shows the results of two kinds of tasks provided by Hadoop, TestDFSIO and EstimatePI. Compared to failover, applications experience much fewer performance declines during live migration. For the throughput of DFSIO, there is only a 12.5% throughput loss when using `MigrRDMA` to migrate the tasks. In contrast, for failover, the throughput loss is up to 65.8%. In terms of the JCT of DFSIO and EstimatePI, there are only the extra 3 seconds for RDMA live migration, versus the extra 20 seconds the failover incurs. EstimatePI does not calculate the throughput result. Thus, `MigrRDMA` provides benefits for RDMA-based applications when they need to be migrated to another physical machine.

VI. RELATED WORKS

RDMA virtualization. RDMA virtualization has gained enormous traction in both academia and industry [40], [43], [51]–[56]. Their main focus is resource sharing, isolation, and network virtualization. For example, HyV [55] mainly provides virtualized RDMA I/O for VMs. FreeFlow [51] and MasQ [43] provide RDMA virtualization for virtual private clouds. Justitia [56] achieves fine-grained QoS policies over RDMA through queue virtualization. `MigrRDMA`'s virtualization mainly focuses on hiding the changes in RDMA communications when migrating an RDMA-based application to a new location.

Migration on RDMA-enabled systems. Some other works provide fast migration features by leveraging RDMA. Huang et al. [57] and Wei et al. [58] leverage RDMA to transfer the VM's/container's states to reduce both the migration time and performance impact on hosted applications. `MigrRDMA` focuses on the live migration of applications running RDMA, but uses TCP to transfer the states. It is possible to integrate these works with `MigrRDMA` to achieve faster live migration. There is also research that provides communication abstraction over RDMA, and has considered live migration features. For example, SocksDirect [59] provides sockets over RDMA fabric. As SocksDirect builds an abstraction layer to translate RDMA interface to socket interface, it can naturally virtualize the states inside the abstraction to hide differences, and ensure the consistency of inflight packets. Achieving communication pre-setup is also straightforward – switching the socket to TCP during RDMA pre-setup, and switching it back to the new RDMA communication when it is ready. `MigrRDMA` focuses

on migrating applications directly using the RDMA fabrics, where there are unique challenges to provide lightweight virtualization to hide the changes, ensure inflight consistency, and achieve communication pre-setup to reduce the blackout.

VII. Conclusion

This paper proposes `MigrRDMA`, a readily deployable software-based system to enable RDMA live migration over commodity RNICs. `MigrRDMA` develops several novel mechanisms to tackle the unique challenges of RDMA live migration, namely, a separate RDMA-related memory restoration to enable RDMA pre-setup, lightweight virtualization to hide differences between migration source and destination, and efficient wait-before-stop solution to preserve inflight WR consistency. Evaluations show the great benefit from RDMA pre-setup (nearly 50% optimization compared to the case without RDMA pre-setup) and low overheads of wait-before-stop, and only 2% $\sim$ 9% extra overheads introduced in the data path. The evaluation for real-world applications demonstrates that `MigrRDMA` provides benefits for RDMA-based applications when they need to be migrated to another server, achieving only a 12.5% throughput loss and an extra 3-second JCT.

References

[1] A. Ruprecht, D. Jones, D. Shiraev, G. Harmon, M. Spivak, M. Krebs, M. Baker-Harvey, and T. Sanderson, "VM Live Migration At Scale," in *Proceedings of the 14th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 45–56.

[2] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in *2nd Symposium on Networked Systems Design & Implementation (NSDI 05)*. Boston, MA: USENIX Association, May 2005.

[3] J. Zhao, R. Shu, L. Qu, Z. Yang, N. E. Jerger, D. Chiou, P. Cheng, and Y. Xiong, "SmartNIC-Enabled Live Migration for Storage-Optimized VMs," in *Proceedings of the 15th ACM SIGOPS Asia-Pacific Workshop on Systems*, ser. APSys '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 45–52.

[4] B. Shi, H. Shen, B. Dong, and Q. Zheng, "Memory/Disk Operation Aware Lightweight VM Live Migration," *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1895–1910, 2022.

[5] V. Marmol and A. Tucker, "Task Migration at Scale using CRIU," https://www.slideshare.net/RohitJnagal/task-migration-using-criu, 2018.

[6] J. Im, J. Kim, J. Kim, S. Jin, and S. Maeng, "On-demand Virtualization for Live Migration in Bare Metal Cloud," in *Proceedings of the 2017 Symposium on Cloud Computing*, ser. SoCC '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 378–389.

[7] J. Im, J. Kim, Y. Kwon, and S. Maeng, "On-Demand Virtualization for Post-Copy OS Migration in Bare-Metal Cloud," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 2028–2038, 2023.

[8] A. Gangidi, R. Miao, S. Zheng, S. J. Bondu, G. Goes, H. Morsy, R. Puri, M. Riftadi, A. J. Shetty, J. Yang, S. Zhang, M. J. Fernandez, S. Gandham, and H. Zeng, "RDMA over Ethernet for Distributed Training at Meta Scale," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 57–70.

[9] K. Qian, Y. Xi, J. Cao, J. Gao, Y. Xu, Y. Guan, B. Fu, X. Shi, F. Zhu, R. Miao, C. Wang, P. Wang, P. Zhang, X. Zeng, E. Ruan, Z. Yao, E. Zhai, and D. Cai, "Alibaba HPN: A Data Center Network for Large Language Model Training," in *Proceedings of the ACM SIGCOMM 2024 Conference*, ser. ACM SIGCOMM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 691–706.

[10] Y. Gao, Q. Li, L. Tang, Y. Xi, P. Zhang, W. Peng, B. Li, Y. Wu, S. Liu, L. Yan, F. Feng, Y. Zhuang, F. Liu, P. Liu, X. Liu, Z. Wu, J. Wu, Z. Cao, C. Tian, J. Wu, J. Zhu, H. Wang, D. Cai, and J. Wu, "When Cloud Storage Meets RDMA," in *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, Apr. 2021, pp. 519–533.

[11] W. Bai, S. S. Abdeen, A. Agrawal, K. K. Attre, P. Bahl, A. Bhagat, G. Bhaskara, T. Brokhman, L. Cao, A. Cheema *et al.*, "Empowering Azure Storage with RDMA," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 2023, pp. 49–67.

[12] W. Cao, Y. Liu, Z. Cheng, N. Zheng, W. Li, W. Wu, L. Ouyang, P. Wang, Y. Wang, R. Kuan, Z. Liu, F. Zhu, and T. Zhang, "POLARDB Meets Computational Storage: Efficiently Support Analytical Workloads in Cloud-Native Relational Database," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*. Santa Clara, CA: USENIX Association, Feb. 2020, pp. 29–41.

[13] Y. Zhang, C. Ruan, C. Li, X. Yang, W. Cao, F. Li, B. Wang, J. Fang, Y. Wang, J. Huo, and C. Bi, "Towards Cost-effective and Elastic Cloud Database Deployment via Memory Disaggregation," *Proc. VLDB Endow.*, vol. 14, no. 10, pp. 1900–1912, jun 2021.

[14] M. Planeta, J. Bierbaum, L. S. D. Antony, T. Hoefler, and H. Härtig, "MigrOS: Transparent Live-Migration Support for Containerised RDMA Applications," in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, Jul. 2021, pp. 47–63.

[15] "eRDMA," https://www.alibabacloud.com/help/en/ecs/user-guide/elastic-rdma-erdma/, 2025.

[16] A. Y. Polyakov, G. Shalom, A. Schwartz, A. Yehezkel, O. Ben David, O. Kahalon, A. Shahar, and L. Liss, "Device-Assisted Live Migration of RDMA Devices," in *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, ser. SOSP '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 153–168.

[17] X. Wei, F. Lu, R. Chen, and H. Chen, "KRCORE: A Microsecond-scale RDMA Control Plane for Elastic Computing," in *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. Carlsbad, CA: USENIX Association, Jul. 2022, pp. 121–136.

[18] CRIU, "CRIU Main Page," https://criu.org/Main_Page, 2023.

[19] "runc," https://github.com/opencontainers/runc, 2024.

[20] A. Mirkin, A. Kuznetsov, and K. Kolyshkin, "Containers Checkpointing and Live Migration," in *Proceedings of the Linux Symposium*, vol. 2, 2008, pp. 85–90.

[21] T. Fukai, T. Shinagawa, and K. Kato, "Live Migration in Bare-Metal Clouds," *IEEE Transactions on Cloud Computing*, vol. 9, no. 1, pp. 226–239, 2021.

[22] K. Cheng, S. Doddamani, T.-C. Chiueh, Y. Li, and K. Gopalan, "Directvisor: Virtualization for Bare-metal Cloud," in *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 45–58.

[23] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy Live Migration of Virtual Machines," *SIGOPS Oper. Syst. Rev.*, vol. 43, no. 3, p. 14–26, Jul. 2009.

[24] J. Corbet, "TCP Connection Repair," https://lwn.net/Articles/495304/, 2012.

[25] E. Jeong, S. Wood, M. Jamshed, H. Jeong, S. Ihm, D. Han, and K. Park, "mTCP: A Highly Scalable User-level TCP Stack for Multicore Systems," in *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*. Seattle, WA: USENIX Association, Apr. 2014, pp. 489–502.

[26] M. Marty, M. de Kruijf, J. Adriaens, C. Alfeld, S. Bauer, C. Contavalli, M. Dalton, N. Dukkipati, W. C. Evans, S. Gribble, N. Kidd, R. Kononov, G. Kumar, C. Mauer, E. Musick, L. Olson, E. Rubow, M. Ryan, K. Springborn, P. Turner, V. Valancius, X. Wang, and A. Vahdat, "Snap: A Microkernel Approach to Host Networking," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, ser. SOSP '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 399–413.

[27] B. Zhu, Y. Chen, Q. Wang, Y. Lu, and J. Shu, "Octopus+: An RDMA-Enabled Distributed Persistent Memory File System," *ACM Trans. Storage*, vol. 17, no. 3, aug 2021.

[28] Y. Chen, X. Wei, J. Shi, R. Chen, and H. Chen, "Fast and General Distributed Transactions Using RDMA and HTM," in *Proceedings of the Eleventh European Conference on Computer Systems*, ser. EuroSys '16. New York, NY, USA: Association for Computing Machinery, 2016.

[29] X. Wei, Z. Dong, R. Chen, and H. Chen, "Deconstructing RDMA-enabled Distributed Transactions: Hybrid is Better!" in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. Carlsbad, CA: USENIX Association, Oct. 2018, pp. 233–251.

[30] J. Shi, Y. Yao, R. Chen, H. Chen, and F. Li, "Fast and Concurrent RDF Queries with RDMA-Based Distributed Graph Exploration," in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, pp. 317–332.

[31] H. Chen, C. Li, C. Zheng, C. Huang, J. Fang, J. Cheng, and J. Zhang, "G-Tran: A High Performance Distributed Graph Database with a Decentralized Architecture," *Proc. VLDB Endow.*, vol. 15, no. 11, p. 2545–2558, jul 2022.
[32] S. K. Monga, S. Kashyap, and C. Min, "Birds of a Feather Flock Together: Scaling RDMA RPCs with Flock," in *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, ser. SOSP '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 212–227.
[33] Q. Wang, Y. Lu, and J. Shu, "Sherman: A Write-Optimized Distributed B+Tree Index on Disaggregated Memory," in *Proceedings of the 2022 International Conference on Management of Data*, ser. SIGMOD '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1033–1048.
[34] B. Yan, Y. Lu, Q. Wang, M. Xie, and J. Shu, "Patronus: High-Performance and Protective Remote Memory," in *21st USENIX Conference on File and Storage Technologies (FAST 23)*. Santa Clara, CA: USENIX Association, Feb. 2023, pp. 315–330.
[35] "InfiniBand Architecture Specification Volume 1," https://www.afs.enea.it/asantoro/V1r1_2_1.Release_12062007.pdf, 2007.
[36] Mellanox Technologies, "Mellanox Adapters Programmer's Reference Manual (PRM)," https://network.nvidia.com/files/doc-2020/ethernet-adapters-programming-manual.pdf, 2016.
[37] X. Li, R. Shu, Y. Xiong, and F. Ren, "Software-based Live Migration for RDMA," in *Proceedings of the ACM SIGCOMM 2025 Conference*, ser. SIGCOMM '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 99–113.
[38] Z. Wang, L. Luo, Q. Ning, C. Zeng, W. Li, X. Wan, P. Xie, T. Feng, K. Cheng, X. Geng, T. Wang, W. Ling, K. Huo, P. An, K. Ji, S. Zhang, B. Xu, R. Feng, T. Ding, K. Chen, and C. Guo, "SRNIC: A Scalable Architecture for RDMA NICs," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 1–14.
[39] K. Koh, K. Kim, S. Jeon, and J. Huh, "Disaggregated Cloud Memory with Elastic Block Management," *IEEE Transactions on Computers*, vol. 68, no. 1, pp. 39–52, 2019.
[40] S.-Y. Tsai and Y. Zhang, "LITE Kernel RDMA Support for Datacenter Applications," in *Proceedings of the 26th Symposium on Operating Systems Principles*, ser. SOSP '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 306–324.
[41] J. Lou, X. Kong, J. Huang, W. Bai, N. S. Kim, and D. Zhuo, "Harmonic: Hardware-assisted RDMA Performance Isolation for Public Clouds," in *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. Santa Clara, CA: USENIX Association, Apr. 2024, pp. 1479–1496.
[42] B. Rothenberger, K. Taranov, A. Perrig, and T. Hoefler, "ReDMArk: Bypassing RDMA Security Mechanisms," in *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Aug. 2021, pp. 4277–4292.
[43] Z. He, D. Wang, B. Fu, K. Tan, B. Hua, Z.-L. Zhang, and K. Zheng, "MasQ: RDMA for Virtual Private Cloud," in *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, ser. SIGCOMM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1–14.
[44] Network-Based Computing Laboratory, "High-Performance Big Data (HiBD)," http://hibd.cse.ohio-state.edu/, 2021.
[45] "GitHub - linux-rdma/perftest: Infiniband Verbs Performance Tests," https://github.com/linux-rdma/perftest.
[46] NVIDIA, "Understanding MLX5 Ethtool Counters," https://enterprise-support.nvidia.com/s/article/understanding-mlx5-ethtool-counters, 2022.
[47] X. Kong, J. Chen, W. Bai, Y. Xu, M. Elhaddad, S. Raindel, J. Padhye, A. R. Lebeck, and D. Zhuo, "Understanding RDMA Microarchitecture Resources for Performance Isolation," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 31–48.
[48] L. Liss, "The Linux SoftRoCE Driver," https://www.openfabrics.org/images/eventpresos/2017presentations/205_SoftRoCE_LLiss.pdf, 2017.
[49] Apache Hadoop, https://hadoop.apache.org/, 2021.
[50] N. S. Islam, D. Shankar, X. Lu, M. Wasi-Ur-Rahman, and D. K. Panda, "Accelerating I/O Performance of Big Data Analytics on HPC Clusters through RDMA-Based Key-Value Store," in *2015 44th International Conference on Parallel Processing*, 2015, pp. 280–289.
[51] D. Kim, T. Yu, H. H. Liu, Y. Zhu, J. Padhye, S. Raindel, C. Guo, V. Sekar, and S. Seshan, "FreeFlow: Software-based virtual RDMA networking for containerized clouds," in *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. Boston, MA: USENIX Association, Feb. 2019, pp. 113–126.
[52] L. Liss, "Containing RDMA and High Performance Computing," 2015.
[53] ——, "RDMA Container support," https://www.openfabrics.org/images/eventpresos/workshops2015/DevWorkshop/Tuesday/tuesday_08.pdf, 2015.
[54] Alibaba Container Service, "Using RDMA on Container Service for Kubernetes," https://www.alibabacloud.com/blog/using-rdma-on-container-service-for-kubernetes_594462?spm=a2c41.12560487.0.0, 2019.
[55] J. Pfefferle, P. Stuedi, A. Trivedi, B. Metzler, I. Koltsidas, and T. R. Gross, "A Hybrid I/O Virtualization Framework for RDMA-Capable Network Interfaces," in *Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*, ser. VEE '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 17–30.
[56] Y. Zhang, Y. Tan, B. Stephens, and M. Chowdhury, "Justitia: Software Multi-Tenancy in Hardware Kernel-Bypass Networks," in *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. Renton, WA: USENIX Association, Apr. 2022, pp. 1307–1326.
[57] W. Huang, Q. Gao, J. Liu, and D. K. Panda, "High Performance Virtual Machine Migration with RDMA over Modern Interconnects," in *2007 IEEE International Conference on Cluster Computing*, 2007, pp. 11–20.
[58] X. Wei, F. Lu, T. Wang, J. Gu, Y. Yang, R. Chen, and H. Chen, "No Provisioned Concurrency: Fast RDMA-codesigned Remote Fork for Serverless Computing," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 497–517.
[59] B. Li, T. Cui, Z. Wang, W. Bai, and L. Zhang, "Socksdirect: Datacenter Sockets can be Fast and Compatible," in *Proceedings of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 90–103.

BIOGRAPHY

Xiaoyu Li achieved his bachelor's degree in University of Electronic Science and Technology (UESTC), Chengdu, China, in 2018, and received the Ph.D. degree in Computer Science and Technology from Tsinghua University, Beijing, China, in 2025. He is now a postdoctoral researcher in Pengcheng Laboratory, Shenzhen, China. His research interests are primarily in data center networks, RDMA, and the RDMA network stack. He has published 5 papers in conferences including APNet, ICNP, and SIGCOMM.

Weizhe Zhang (Senior Member of IEEE, Lifetime Member of ACM) received the B.Eng., M.Eng., and Ph.D. degrees in computer science and technology from Harbin Institute of Technology, Harbin, China, in 1999, 2001, and 2006, respectively. He is the Professor of the School of Computer Science and Technology, Harbin Institute of Technology, and the Director of the Cyberspace Security Research Center, Peng Cheng Laboratory, Shenzhen, China. He has published over 130 papers in journals, books, and conference proceedings, interested in cyberspace security, cloud computing, and HPC.

Fengyuan Ren is now a professor of Department of Computer Science and Technology in Tsinghua University, Beijing, China. His research interests include network architecture and traffic management, data center network, industrial Internet and system performance evaluation. He has published more than 200 academic papers, and won 6 national, provincial or ministerial scientific and technological awards, supported by the "New Century Talent Support Plan of Ministry of Education" and the "Distinguished Youth Fund Project of National Natural Science Foundation of China".